

Azure DevOps Pipelines

Jobs, Agents, Parallelism & Multi-Stage YAML

Interview ke liye Student Notes | Part 1 – Samajh • Part 2 – Q&A

Pichhle notes mein humne pipeline ka basic dekha tha. Ab thoda **aage ki baat** — ek pipeline ke andar **kai jobs, kai agents, parallel execution**, aur ek poora **multi-stage pipeline** (Build → Scanning → Deploy). Ye wahi cheezein hain jo interviewer round 2 mein deep poochhta hai.

Pehle ye yaad rakho: Pipeline ek automation workflow hai — uske andar stages, stage ke andar jobs, aur job ke andar steps/tasks hote hain. Jobs agents par chalte hain, aur agents ek pool se aate hain. Bas yahi base hai, isi par sab khada hai.

PART 1 — Samajhne Ke Liye (Hinglish)

1. Pipeline ka dhaancha — Stages → Jobs → Steps

Pipeline andar se teen level mein tooti hoti hai. Bade se chhota:

- **Stage** — bada phase. Jaise Build, Scanning, Deploy.
- **Job** — stage ke andar ka kaam, jo ek agent par chalta hai. Ek stage mein kai jobs ho sakte hain.
- **Step / Task** — sabse chhoti cheez. Ek actual command, jaise Terraform install karo, init karo.

Zaroori baat: Agar tum YAML mein sirf steps likho (job na likho), to bhi wo ek default job ke andar, ek agent par hi chalte hain. Matlab job hamesha hota hai — chahe tum likho ya na likho.

2. Job kya hai aur uske fayde

Job matlab **steps/tasks ka ek group**. Har job ko pool se ek agent milta hai. Ek pipeline mein kai jobs alag-alag agents par chal sakte hain.

Jobs alag-alag rakhne ke 5 fayde (interview mein ye bol dena):

1. **Multiple agents** — alag jobs alag machines par chal sakte hain.
2. **Parallel execution** — bina dependency wale jobs ek saath chal jaate hain, time bachta hai.
3. **Dependencies** — ek job doosre ke baad chale, ye set kar sakte ho (dependsOn).
4. **Log readability** — har job ka apna log, dhoondhna aasan.
5. **Isolation** — ek job mein dikkat aaye to doosre par asar nahi padta.

3. Agent aur Pool

- **Agent** — wo machine (VM/server/container) jahaan job actually chalti hai.
- **Pool** — agents ka group. YAML mein pool: self-hosted-pool likhte hain.

Pool override rule (interview favourite): agar job ke level par pool diya hai, to wo pipeline-level wale pool ko **override** kar deta hai. Aur agar job par koi pool nahi diya, to wo job pipeline-level wale pool par chal jaata hai.

Ek special pool — 'server': Manual validation wala job kisi machine par nahi chalta (agentless job hota hai). Isliye uska pool 'server' likhte hain. Ye baat interview mein zaroor poochhi jaati hai.

4. Parallelism — jobs ek saath kab chalte hain

Parallelism matlab — kai jobs ek hi waqt par chalna. Ye tab hota hai jab do jobs ke beech **koi dependency na ho**.

Asaan misaal — ek stage mein 4 kaam hain:

Agar **ek ke baad ek (sequential)** chale: Init+Plan (5 min) → Scanning (20 min) → Approval → Apply (5 min). Scanning sabko rok ke baithi rehti hai.

Agar Scanning aur Init+Plan ke beech dependency na ho, to wo **parallel** chal sakte hain — total time kam ho jaata hai.

Kaise enable karein: jobs ko alag-alag rakho aur unke beech dependsOn mat lagao — Azure DevOps unhe khud parallel chala dega (jitne parallel agents available honge). Sequence chahiye to dependsOn laga do.

Fayde: time bachta hai, agents ka acha use hota hai, aur bade deployments tez ho jaate hain.

5. Real example — Multi-region / Multi-subscription deploy

Maan lo infra **teen region (ya teen subscription)** mein deploy karna hai. Har ek ke liye wahi steps: Init+Plan → Manual Approval → Apply.

Agar ek-ek karke karo to teen guna time. Isliye har region/subscription ka apna **alag job** banate hain aur **multiple agents** par **parallel** chalte hain — Region 1 Agent 1 par, Region 2 Agent 2 par, saath-saath.

Interview line: 'For multi-region or multi-subscription deployments, I create a separate job per region and run them in parallel on multiple agents, so the total deployment time stays low instead of multiplying.'

6. Multi-stage pipeline ka structure (Build → Scanning → Deploy)

Yahi structure aaj kal real projects mein use hota hai. Teen stage, har stage mein jobs:

Stage 1 — Build

- Job: **Terraform Fmt & Validate** — code ka format aur syntax check.
- Job: **Terraform Init & Plan** — backend set karo, phir plan banao (kya banega ye dikhao).

Stage 2 — Scanning

- Job: **TFSec** — Terraform code mein security problems dhoondhta hai.
- Job: **TFLint** — Terraform code ke style/mistakes check karta hai.

Stage 3 — Deploy

- Job: **Manual Validation** — pool 'server', insaan approve karta hai.
- Job: **Terraform Apply** — dependsOn: ManualValidation, approve hote hi infra ban jaata hai.

Deploy stage ka chalu hona condition par: Deploy stage tabhi chale jab branch main ho — isliye uspe condition lagate hain. Feature branch par sirf Build aur Scanning chalti hai, deploy nahi. Isse galti se feature code production mein nahi jaata.

7. Parameters, Variables aur Conditions

Parameter — pipeline chalate waqt diya jaane wala input. Jaise environment (dev/prod) ya ScanningRequired (true/false). User dropdown se choose karta hai.

Variable — andar set ki gayi value. Jaise workDir jo parameter ke hisaab se folder ka raasta banata hai: environments/dev ya environments/prod.

Condition — kaam tabhi chale jab koi shart sach ho. Do common:

- Branch condition — Deploy sirf main par: condition: `eq(variables['Build.SourceBranch'], 'refs/heads/main')`
- Parameter condition — Scanning tabhi jab user 'true' chune: condition: `eq('${ parameters.ScanningRequired }', 'true')`

8. Terraform pipeline ke steps (sahi order)

6. **Install** — Terraform install karo.
7. **Init** — backend (Azure Storage) ke saath set up, state file yahan rehti hai.
8. **Fmt** — code ko saaf format karo.
9. **Validate** — syntax aur config sahi hai ya nahi check karo.
10. **Plan** — kya badlega uska preview banao.
11. **Manual Validation** — insaan plan dekh ke approve kare.
12. **Apply** — actual infra bana do.

Apply stage mein init dobara kyun? Plan aur Apply alag stage/agent par chalte hain, aur beech mein Terraform ka cache flush ho jaata hai. Isliye Apply se pehle Terraform init dobara karna padta hai (wahi backend ke saath) — warna error aata hai.

9. Kaam ke Linux / Mac commands (agent par)

- `brew install <tool>` — Mac par software install (package manager).
- `touch file.txt` — khaali file banao / timestamp update.
- `ls -la` — files dikhao (hidden samet, lambi list).
- `mkdir project` — nayi folder banao.
- `rm file.txt / rm -r folder` — file / folder delete.

10. Poora sample YAML (Build → Scanning → Deploy)

Ye saaf, professional pipeline hai — interview mein dikhane/samjhane ke liye perfect:

```
trigger:
  branches:
    include:
      - main
      - feature/*

pool: self-hosted-pool

parameters:
  - name: environment
    displayName: Environment (dev/prod)
    type: string
    default: dev
    values:
      - dev
      - prod

variables:
  workDir: '$(System.DefaultWorkingDirectory)/environments/${{ parameters.environment }}'

stages:
  - stage: Build
    displayName: Build
    jobs:
      - job: FmtValidate
        displayName: Terraform Fmt and Validate
        steps:
          - task: TerraformInstaller@1
            inputs:
              terraformVersion: 'latest'
          - task: TerraformTask@5
            inputs:
              provider: 'azurerm'
              command: 'validate'
              workingDirectory: $(workDir)

      - job: InitPlan
        displayName: Terraform Init and Plan
        steps:
          - task: TerraformInstaller@1
            inputs:
              terraformVersion: 'latest'
          - task: TerraformTask@5
            inputs:
              provider: 'azurerm'
              command: 'init'
              workingDirectory: $(workDir)
              backendServiceArm: 'azure-connection'
              backendAzureRmStorageAccountName: 'tfstatestore'
              backendAzureRmContainerName: 'tfstate'
              backendAzureRmKey: '${{ parameters.environment }}.terraform.tfstate'
          - task: TerraformTask@5
            inputs:
              provider: 'azurerm'
              command: 'plan'
              workingDirectory: $(workDir)
```

```

environmentServiceNameAzureRM: 'azure-connection'

- stage: Scanning
  displayName: Scanning
  jobs:
    - job: TFSecScan
      displayName: TFSec Security Scan
      steps:
        - task: tfsec@1
          inputs:
            version: 'v1.26.0'
            dir: $(workDir)
    - job: TFLintScan
      displayName: TFLint Scan
      steps:
        - script: tflint --chdir=$(workDir)
          displayName: 'Run TFLint'

- stage: Deploy
  displayName: Deploy
  condition: eq(variables['Build.SourceBranch'], 'refs/heads/main')
  jobs:
    - job: ManualValidation
      displayName: Manual Validation
      pool: server
      steps:
        - task: ManualValidation@1
          inputs:
            notifyUsers: 'team-lead@company.com'
            instructions: 'Please review the Terraform plan before approving.'
    - job: TerraformApply
      displayName: Terraform Apply
      dependsOn: ManualValidation
      steps:
        - task: TerraformTask@5
          inputs:
            provider: 'azurerm'
            command: 'init'
            workingDirectory: $(workDir)
            backendServiceArm: 'azure-connection'
            backendAzureRmStorageAccountName: 'tfstatestore'
            backendAzureRmContainerName: 'tfstate'
            backendAzureRmKey: '${{ parameters.environment }}.terraform.tfstate'
        - task: TerraformTask@5
          inputs:
            provider: 'azurerm'
            command: 'apply'
            workingDirectory: $(workDir)
            environmentServiceNameAzureRM: 'azure-connection'

```

Padhne ka tareeka: Upar se neeche — pehle trigger (kab chale), phir pool (kahaan chale), phir parameters/variables (kya input), phir stages. Har stage ke andar jobs, har job ke andar steps. Bas isi tarah class mein line-by-line padha dena.

PART 2 — Interview Q&A (English)

Niche har jawab simple English mein hai, aur uske neeche ek **Matlab (Hindi mein)** line — taaki samajh aaye kya bolna hai. Apne project ke saath bolo.

Q1. What is the structure of an Azure DevOps pipeline?

A pipeline has **stages**, each stage has **jobs**, and each job has **steps or tasks**. Stages are big phases like Build, Scanning, and Deploy; jobs run on agents; steps are the actual commands.

Matlab (Hindi mein): pipeline ka order bata rahe ho — stages > jobs > steps. Bade se chhota.

Q2. What is the difference between a job and a step?

A **step** is a single task, like installing Terraform. A **job** is a group of steps that runs together on one agent. Even if you write only steps, they run inside a default job.

Matlab (Hindi mein): step ek kaam hai, job kai steps ka group jo ek agent par chalta hai. Steps hamesha kisi job ke andar hi chalte hain.

Q3. What is an agent and what is a pool?

An **agent** is the machine where a job actually runs. A **pool** is a group of agents. In YAML we point a job or the whole pipeline to a pool.

Matlab (Hindi mein): agent wo machine hai jahaan job chalti hai; pool agents ka group hai.

Q4. If a job has its own pool, which pool is used?

The job-level pool **overrides** the pipeline-level pool. If a job doesn't set a pool, it uses the pipeline-level pool.

Matlab (Hindi mein): job ka apna pool pipeline wale pool ko override kar deta hai; job par pool na ho to pipeline ka pool chalta hai.

Q5. What is the 'server' pool used for?

It's used for **agentless jobs** like Manual Validation, which don't run on a machine. So we set `pool: server` for the manual approval job.

Matlab (Hindi mein): manual validation kisi machine par nahi chalta, isliye uska pool 'server' likhte hain.

Q6. What is parallelism in pipelines? When can jobs run in parallel?

Parallelism means running **multiple jobs at the same time**. Jobs can run in parallel when there is **no dependency** between them.

Matlab (Hindi mein): parallelism matlab kai jobs ek saath chalna; ye tab hota hai jab unke beech koi dependency na ho.

Q7. How do you enable parallel jobs and what are the benefits?

Keep the jobs separate and **don't add dependsOn** between them — Azure DevOps runs them in parallel on available agents. The benefits are less total time, better use of agents, and faster large deployments.

Matlab (Hindi mein): jobs alag rakho aur beech mein dependsOn mat lagao — wo apne aap parallel chal jaayenge. Fayda: time bachta hai.

Q8. How do you make jobs run in a specific order?

I use `dependsOn` on a job (or stage) to point to the one that must finish first. For example, the `Apply` job has `dependsOn: ManualValidation`.

Matlab (Hindi mein): `dependsOn` lagao taaki ek job doosre ke khatam hone ke baad hi chale.

Q9. What is the difference between a parameter and a variable?

A **parameter** is an input you choose when starting the pipeline, like environment `dev` or `prod`. A **variable** is a value set inside the pipeline, like the working directory built from that parameter.

Matlab (Hindi mein): parameter wo input jo tum chalte waqt dete ho; variable andar set ki gayi value hoti hai.

Q10. How do you make a stage run only on the main branch?

I add a condition on the stage: `condition: eq(variables['Build.SourceBranch'], 'refs/heads/main')`. So `Deploy` runs only on `main`, while feature branches run only `Build` and `Scanning`.

Matlab (Hindi mein): stage par branch condition lagate ho, taaki `Deploy` sirf `main` branch par chale, feature par nahi.

Q11. How do you make a job run only when needed (conditionally)?

I use a condition based on a parameter. For example, the `Scanning` job runs only when `ScanningRequired` is set to `true`.

Matlab (Hindi mein): parameter par condition lagate ho — jaise `scanning` tabhi chale jab user `'true'` chune.

Q12. What are TFSec and TFLint, and why use them?

TFSec scans Terraform code for security problems, and **TFLint** checks for mistakes and bad style. We run them in a `Scanning` stage to catch issues before deploying.

Matlab (Hindi mein): `TFSec` security problems pakadta hai, `TFLint` code ki galtiyan/style. `Deploy` se pehle scan kar lete hain.

Q13. What do terraform fmt and validate do?

fmt formats the code neatly, and **validate** checks that the configuration is syntactically correct. Both run early in the `Build` stage to catch problems before `plan` and `apply`.

Matlab (Hindi mein): `fmt` code ko saaf format karta hai, `validate` syntax sahi hai ya nahi check karta hai.

Q14. Why do you run terraform init again before apply?

Because `plan` and `apply` run on different stages/agents and the Terraform cache is cleared in between. So before `apply` I run `init` again with the same backend, otherwise it errors out.

Matlab (Hindi mein): `plan` ke baad cache flush ho jaata hai, isliye `apply` se pehle `init` dobara karna padta hai warna error aata hai.

Q15. How do you deploy infrastructure to multiple regions or subscriptions efficiently?

I create a **separate job for each region or subscription** and run them **in parallel on multiple agents**, instead of one after another. This keeps the total time low.

Matlab (Hindi mein): har region/subscription ka alag job banate ho aur multiple agents par parallel chalte ho — time guna nahi hota.

Q16. What are the advantages of splitting work into multiple jobs?

Multiple agents can be used at once, jobs can run in parallel, you can set dependencies, logs are easier to read per job, and one failing job doesn't break the others (isolation).

Matlab (Hindi mein): multiple agents, parallel chalna, dependency set karna, har job ka alag log, aur isolation — ek job fail ho to baaki par asar nahi.

Quick Revision Tips

- **Hierarchy ratto mat, samjho:** Stages → Jobs → Steps. Jobs agents par, agents pool se.
- **Do override rule yaad:** job ka pool > pipeline ka pool. Manual validation = pool 'server'.
- **Parallel = no dependency.** Sequence chahiye to dependsOn lagao.
- **Deploy sirf main par** (branch condition); Scanning optional (parameter condition).
- **Apply se pehle init dobara** — ye chhoti baat interview mein bada impression deti hai.
- **Ek hi project se baat karo** aur 'I' use karo — practical lagta hai.

Practice karo, bol-bol ke. All the best!